Journal of Ambient Intelligence and Humanized Computing (2023) 14:1675–1698
https://doi.org/10.1007/s12652-021-03388-2

ORIGINAL RESEARCH

Joint optimization of energy consumption and time delay in IoT-fog-cloud computing environments using NSGA-II metaheuristic algorithm

Vahid Jafari[1] · Mohammad Hossein Rezvani[1]

Received: 25 December 2020 / Accepted: 6 July 2021 / Published online: 23 July 2021
© The Author(s), under exclusive licence to Springer-Verlag GmbH Germany, part of Springer Nature 2021

Abstract
Today, there exists a growing demand for Internet of Things (IoT) services in the form of vehicle networks, smart cities, augmented reality, virtual reality, positioning systems, and so on. Due to the considerable distance between the IoT devices and the central cloud, using this option may no longer be a suitable solution for delay-constraint tasks. To overcome these drawbacks, a complementary solution called fog computing, also known as the *cloud at the edge* is used. In this solution, nodes at the edge of the network provide resources for IoT applications. Although offloading tasks on the fog nodes save energy on IoT devices, it increases task response time. Therefore, making a trade-off between energy consumption and latency is crucial for IoT devices. Because offloading falls into the category of NP-hard knapsack problems, metaheuristic methods have been widely used in recent years. In this paper, we formulate the problem of joint optimization of energy consumption and latency in the form of a multi-objective problem and solve it using the non-dominant sorting genetic algorithm (NSGA-II) and Bees algorithm (BA). Also, to improve the quality of solutions, we combine each of these methods with a robust type of differential evolution approach called minimax differential evolution (MMDE). This combination moves the solutions to better areas and increases the convergence speed. The simulation results show that NSGA-based methods have remarkable robustness compared to BA-based methods in terms of significant criteria such as energy consumption, time delay, and so on. Our statistical analysis shows that both NSGA-based and BA-based metaheuristic methods not only do not significantly increase energy consumption but also drastically reduce response time.

Keywords Internet of Things (IoT) · Fog computing · Task offloading · Energy consumption · Time-constraint task · Genetic algorithm

1 Introduction

Today, the Internet of Things (IoT) includes data generated from smartphones, IoT devices, vehicular networks, smart cities, augmented reality (AR), virtual reality (VR), and positioning systems (Salaht et al. 2020; Caizaet al. 2020). On the other hand, the number of IoT devices connected to the Internet around the world is constantly growing, which in turn requires more cloud processing and storage resources. Given that the traditional cloud is usually far away from IoT devices, it may not be a suitable solution for delay-constrained applications (Keshavarznejad et al. 2021; Millham et al. 2021). To overcome these drawbacks, fog computing has been introduced by Cisco as a complementary solution to the cloud (Buyya et al. 2019; Mebrek et al. 2017). In this solution, nodes at the edge of the network provide applications with processing and storage resources. Fog computing provides high scalability because it brings both computing and radio sources closer to IoT devices. Although offloading tasks to fog can save energy on IoT devices, it causes more delays. This time delay mainly includes the waiting time in the fog and the communication time between the fog and the IoT devices. Therefore, making a trade-off between energy consumption and time delay efficiency is crucial for IoT devices (Keshavarznejad et al. 2021).

✉ Mohammad Hossein Rezvani
rezvani@qiau.ac.ir

Vahid Jafari
vahid.jafari@qiau.ac.ir

[1] Faculty of Computer and Information Technology Engineering, Qazvin Branch, Islamic Azad University, Qazvin, Iran

So far, extensive research have been conducted on the joint minimization of delay and energy consumption.

To solve the problem, some researchers have used classic mathematical methods (Sun et al. 2020; Yang et al. 2018; Liu et al. 2018; Chiti et al. 2018; Zhang et al. 2019; Shahidinejad and Ghobaei-Arani 2020; Jazayeri et al. 2021). Since task offloading falls into the category of NP-hard knapsack problems (Keshavarznejad et al. 2021), some researchers have used heuristic methods to reduce temporal complexity (Besharati and Rezvani 2019; Dinh et al. 2021; Yassine et al. 2019; De Maio and Kimovski 2020; Elashri and Azim 2020). Other researchers have also used metaheuristic methods to solve the optimization problem (Keshavarznejad et al. 2021; Huang et al. 2019; Hussein and Mousa 2020; Subramaniam and Krishnasamy 2021; Adhikari et al. 2019; Manasrah and Gupta 2019; Parvizi and Rezvani 2020; Ghobaei-Arani et al. 2020; Djemai et al. 2019; Aboutorabi and Rezvani 2020; Tavakoli-Someh and Rezvani 2019; Abbasi et al. 2020; Mohammadi and Rezvani 2019). Of course, in these methods, reducing the delay requires achieving a near-optimal solution, rather than an optimal one.

In this paper, we aim to make a trade-off between time delay and energy consumption. To this end, we solve the problem of joint minimizing power consumption and time delay for IT devices in a fog computing environment. The main contributions of this research are as follows:

- We formulate the problem of joint optimization of energy consumption and delay in the form of a multi-objective problem. In most previous research, the Poisson distribution has been used to model the incoming traffic of IoT users. The Poisson distribution may not reflect the actual behavior of users. In this study, we propose a model for task offloading that does not depend on specific assumptions at all. Our model has no particular dependence on queuing theory assumptions.
- Many of the previous works did not consider the direct user-cloud offloading to avoid complexities in modeling. We consider both user-to-cloud and fog-to-cloud cases in our formulation.
- We have considered both the decision cost and the execution cost. For each task, the output of the decision phase specifies to which fog/cloud device it should be sent. Since the offloading decision is an NP-hard problem, we solve it with several optimization methods. The most important methods analyzed are the non-dominated sorting genetic algorithm (NSGA-II) (Deb et al. 2002; Kaur and Kumar 2018) and Bee Algorithm (Keshavarznejad et al. 2021). We also use differential evolution methods (Kaur et al. 2020a) in combination with each metaheuristic method to improve the quality of the solution.

This issue has been investigated in some studies such as Mebrek et al. (2017) using single-objective metaheuristic methods. However, to the best of our knowledge, no research has so far used a multi-objective meta-heuristic method to solve the problem.

The remainder of this paper is organized as follows: Sect. 2 reviews the most important studies on the literature; Sect. 3 explains the problem formulation; Sect. 4 elaborates the proposed method; Sect. 5 evaluates the results of simulating the proposed method using iFogSim (Gupta et al. 2017); Finally, Sect. 6 concludes the study and provides suggestions for future research.

2 Related work

So far, a great body of research has been done about offloading with the aim of joint minimizing energy and execution delay. Before reading this section, interested readers are advised to refer to Salaht et al. (2020), Caiza et al. (2020), Millham et al. (2021), Shakarami et al. (2020) and Lahmar and Boukadi (2020) for a comprehensive review. After studying previous research, we divided them into three categories: classic mathematical methods, heuristic, and metaheuristic ones.

Partial offloading is a new topic in fog computing. Other research activities in this field can be found in (Maity and Mistry 2020; Ning et al. 2020; Wang et al. 2020). Also, Shah-Mansouri and Wong (2018) developed a competitive offloading game among IoT users to solve this problem. Interested readers can refer to Sun et al. (2020), Yang et al. (2018), Liu et al. (2018), Chiti et al. (2018), Zhang et al. (2019), Shahidinejad and Ghobaei-Arani (2020) and Jazayeri et al. (2021) to see other research on classic mathematical methods. There are few studies in which only one goal has been optimized. For example, the goal in Chiti et al. (2018) is only to optimize latency. Conversely, in Zhang et al. (2019), the authors' goal is to optimize energy. The main drawback of these methods is that they are not practical when the number of nodes is large. As mentioned earlier, the offloading problem falls into the category of NP-hard problems (Keshavarznejad et al. 2021), and, hence, to solve it on a large scale, heuristic or metaheuristic methods must be used. In the following, we will explain the research activities on these methods.

In the literature, a major part of offloading research is related to heuristic methods. Liu et al. (2018) solved the offloading optimization problem using a heuristic method called the Interior Point Method (IPM). They aim to obtain the optimal offloading probability and the required transmission power. Like classical mathematical methods, in heuristic methods, some research activities are devoted to optimizing only one goal. Some of them optimize latency

(Yu et al. 2018) while others optimize energy (Jiang et al. 2019). Interested readers can refer to Besharati and Rezvani (2019), Dinh et al. (2021), Yassine et al. (2019), De Maio and Kimovski (2020) and Elashri and Azim (2020) to see other research on heuristic methods.

Some researchers have used metaheuristic methods. The results of these research show that meta-heuristic approaches can achieve the goal with minimal delay. Huang et al. (2019) proposed an Ant Colony Optimization (ACO) algorithm to maximize revenue for cloud service providers and minimize resource utilization. A similar study was conducted by Hussein and Mousa (2020), this time with the aim of load balancing. Subramaniam and Krishnasamy (2021) used the Gray Wolf Optimization (GWO) algorithm to minimize power consumption considering the user's workload. Their results showed that the GWO task offloading method is more efficient than other methods. Despite all the recent attention to computation in the fog layer, due to limited resources, this layer is not suitable for processing CPU-bounded tasks. To address this, Adhikari and Gianey (2019), inspired by the Firefly algorithm, sought joint minimizing of computation time and power consumption. In addition to computational sources, they used CO_2 emissions and temperature emissions as parameters. Adhikari et al. (2019) aim to simultaneously maximize utilization and minimize task processing costs. To do this, they used Particle Swarm Optimization (PSO) approach to reduce computational latency. Interested readers can refer to Manasrah and Gupta (2019), Parvizi and Rezvani (2020), Ghobaei-Arani et al. (2020), Djemai et al. (2019), Aboutorabi and Rezvani (2020), Tavakoli-Someh and Rezvani (2019), Abbasi et al. (2020) and Mohammadi and Rezvani (2019) to see other research on metaheuristic methods.

3 Problem formulation

As shown in Fig. 1, we assume that the system consists of IoT devices, fog nodes, and a central cloud. Each IoT device can only be connected to a fog node via the Base Station (BS). Each IoT device generates some service requests. We denote the set of service requests by $\mathcal{I}$. These requests are entered into the system in the form of objects. If the resources on the IoT device are not sufficient to perform a request, it is offloaded to a fog node. Each of these fog nodes is called a fog instance. Let $\mathcal{F}$ denotes the set of fog instances. Similarly, if the fog node resources are insufficient, the request is offloaded to a central cloud $c \in \mathcal{C}$. The notations used in this research are shown in Table 1.

3.1 Energy consumption

We start with a request $i \in \mathcal{I}$. This request is made by an IoT device and is offloaded to a fog node $f \in \mathcal{F}$. The energy consumed during this process can be divided into three parts: (a) the energy consumed for transmission via BS, routers, intermediate switches, and the technology used in the access network of the fog device f which we denote by $E_{i,f}^{IoT-fog}$ (b) the energy consumed to process and store requests within the fog device (fog instance) which is denoted by $E_{i,fog}^{prc\&str}$; and (c) if the local resources of the fog instance are insufficient, it will offload the request to a central cloud $c \in \mathcal{C}$. We denote the energy consumption in this case by $E_{i,f}^{fog-cloud}$. Total energy consumption in the fog instance $f \in \mathcal{F}$ for the request $i \in \mathcal{I}$ is obtained as follows:

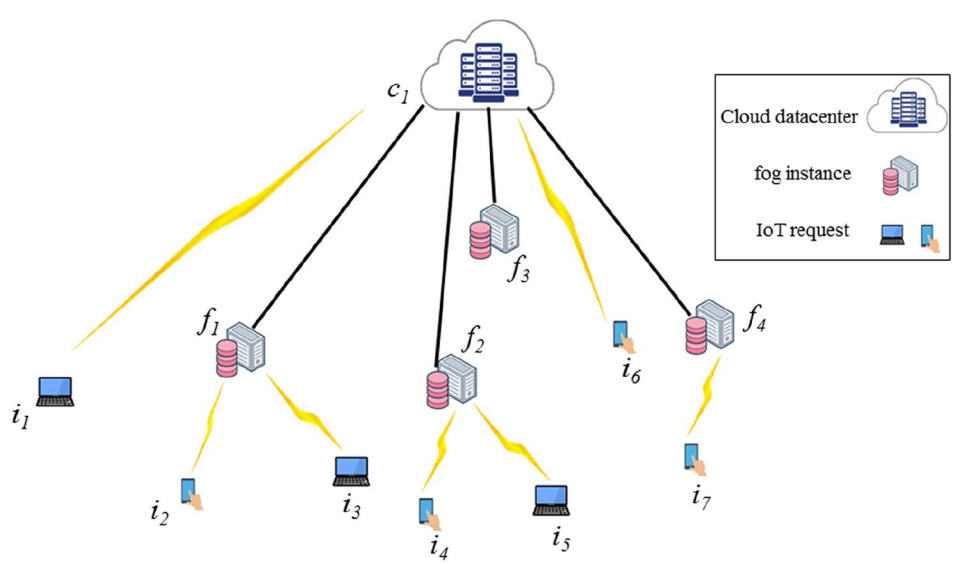

Fig. 1 An example of the environment model

Table 1 Notations used in this research

Symbol	Symbol description
$\mathcal{I}$	The set of all requests
$\mathcal{F}$	The set of all fog instances
$\mathcal{C}$	The set of cloud datacenters
i	IoT device (IoT instance) identifier ($i \in \mathcal{I}$)
f	Fog device (fog instance) identifier ($f \in \mathcal{F}$)
c	Cloud datacenter identifier ($c \in \mathcal{C}$)
N	Number of IoT requests
F	Number of fog instances
M	Number of cloud datacenters
$E_{i,f}^{IoT-fog}$	The consumed energy to offload an object from the IoT instance i to the fog instance f (J)
$E_{i,fog}^{prc\&str}$	The consumed energy for processing and storing an object of the IoT instance i inside a fog instance (J)
$E_{i,f,c}^{fog-cloud}$	The consumed energy to offload an object i from the fog instance f to the cloud datacenter c (J)
$E_{i,f}^{fog-IoT}$	The consumed energy to send the answer from the fog instance f back to the IoT instance i (J)
α	The ratio of requests offloaded by a fog instance to the cloud
$E_{i,f}^{fog}$	The total consumed energy in a fog instance for each request (J)
$E_{i,c}^{IoT-cloud}$	The consumed energy for direct offloading of an object from the IoT instance i to the cloud datacenter c (J)
$E_{i,c}^{prc\&str}$	The consumed energy for processing and storing an object of the IoT instance i inside the cloud datacenter c (J)
$E_{i,c}^{cloud-IoT}$	The consumed energy to send the answer from the cloud data center c back to the IoT instance i (J)
$E_{i,c}^{cloud}$	The total consumed energy in the cloud datacenter c for each request (J)
$E_{network}^{IoT-fog}$	The consumed energy to transmit each byte of request from an IoT instance to a fog instance (J)
$E_{network}^{IoT-cloud}$	The consumed energy to transmit each byte of request from an IoT instance to the cloud datacenter (J)
$h_{IoT-fog}$	the gain of the channel between an IoT instance and the base station
$h_{IoT-cloud}$	the gain of the channel between an IoT instance and the cloud
L_i	The size of a request i (bytes)
L_i'	The size of the answer to a request i (bytes)
C_{idle}^{fog}	The idle capacity of a fog instance (bytes/s)
C_{idle}^{cloud}	The idle capacity of the cloud datacenter (bytes/s)
$C_{\max}^{fog}$	The maximum capacity of a fog instance (bytes/s)
$C_{\max}^{cloud}$	The maximum capacity of the cloud datacenter (bytes/s)
u^{fog}	The utilization of a fog instance
u^{cloud}	The utilization of the cloud datacenter
P_{idle}^{fog}	The idle power consumption of a fog instance (kW)
P_{idle}^{cloud}	The idle power consumption of the cloud datacenter (kW)
P_{busy}^{fog}	The busy power consumption of a fog instance (kW)
P_{busy}^{cloud}	The busy power consumption of the cloud datacenter (kW)
E_{fog}^{prc}	The consumed energy for processing each byte of request inside a fog instance (J)
E_{cloud}^{prc}	The consumed energy for processing each byte of request inside the cloud datacenter (J)
E_{fog}^{str}	The consumed energy to store each byte of request inside a fog instance (J)
E_{cloud}^{str}	The consumed energy to store each byte of request inside the cloud datacenter (J)
β_{idle}	The ratio of the idle period to the total period for each processor inside the fog/cloud device
$E^{fog-cloud}$	The consumed energy to offload each byte of data from a fog instance to the cloud (J)
E_i	The power consumption for an IoT request $i \in \mathcal{I}$
$\tilde{E}$	The aggregate energy consumption of all IoT requests
τ^{fog}	The time duration of each instruction inside a fog instance (s)
θ^{fog}	The size of each instruction inside a fog instance (byte)
τ^{cloud}	The time duration of each instruction inside the cloud datacenter (s)
θ^{cloud}	The size of each instruction inside the cloud datacenter (byte)

Table 1 (continued)

Symbol	Symbol description
$t_{i,f}^{IoT-fog}$	The transmission latency to offload an object from the IoT instance i to the fog instance f (s)
$t_{i,fog}^{prc}$	The processing latency of a request i inside a fog instance (s)
$t_{i,f,c}^{fog-cloud}$	The delay to offload an object i from the fog instance f to the cloud datacenter c (s)
$t_{i,f}^{fog}$	The total delay in a fog instance for each request (s)
$t_{network}^{IoT-fog}$	The transmission delay of each byte of request from an IoT instance to a fog instance (s)
$t_{network}^{IoT-cloud}$	The transmission delay of each byte of request from an IoT instance to the cloud datacenter (s)
$t_{network}^{fog-cloud}$	The transmission delay of each byte of request from a fog instance to the cloud datacenter (s)
$t_{i,c}^{IoT-cloud}$	The transmission latency for direct offloading of an object from the IoT instance i to the cloud datacenter c (s)
$t_{i,c}^{prc}$	The processing latency of a request i inside the cloud datacenter c (s)
$t_{i,c}^{cloud}$	The total delay in the cloud datacenter c for each request (s)
$t_{i}^{deadline}$	The deadline of a request i (s)
t_i	The delay for an IoT request $i \in \mathcal{I}$
$\tilde{t}$	The aggregate delay of all IoT requests
$x_{i,f} \in \{0,1\}$	A decision variable indicating whether an IoT request $i \in \mathcal{I}$ is offloaded to a fog instance $f \in \mathcal{F}$
$\mathbf{X}_{fog}$	The vector containing all decision variables $x_{i,f}$
$x_{i,c} \in \{0,1\}$	A decision variable indicating whether an IoT request $i \in \mathcal{I}$ is offloaded to a cloud datacenter $c \in \mathcal{C}$
$\mathbf{X}_{cloud}$	The vector containing all decision variables $x_{i,c}$
m_i	The demanded memory of an IoT request $i \in \mathcal{I}$
m_f	The maximum affordable memory by a fog instance $f \in \mathcal{F}$
m_c	The maximum affordable memory by a cloud datacenter $c \in \mathcal{C}$

$$E_{i,f}^{fog} = E_{i,f}^{IoT-fog} + E_{i,fog}^{prc\&str} + \alpha . E_{i,f,c}^{fog-cloud} + E_{i,f}^{fog-IoT}, \quad \forall i \in \mathrm{I} \;\; \exists f \in \mathrm{F} \;\; \exists c \in \mathrm{C}, \tag{1}$$

in which $E_{i,f}^{fog-IoT}$ and α are the energy consumed in the fog to send a response back to the object, and the ratio of offloaded requests, respectively.

In our proposed scenario, we assume that each IoT device can stream the request directly to the central cloud. We will explain this operation in detail later in Eq. (6).

We now proceed to calculate $E_{i,f}^{IoT-fog}$ in Eq. (1). We write (Mebrek et al. 2017):

$$E_{i,f}^{IoT-fog} = L_i . E_{network}^{IoT-fog} + L_i . h_{IoT-fog} . \left(\frac{P_{idle}^{fog}}{C_{idle}^{fog}} + \frac{P_{\max}^{fog}}{\omega^{fog} . C_{\max}^{fog}} \right), \tag{2}$$

which L_i denotes the size of the request i in bytes. Also, $E_{network}^{IoT-fog}$ denotes the consumed energy to transmit each byte of request from an IoT instance to a fog instance. Here, $h_{IoT-fog}$ denotes the gain of the channel between an IoT instance and the BS. Also, P_{idle}^{fog} and $P_{\max}^{fog}$ represent the idle power consumption and the maximum power consumption of a fog instance, respectively. We now obtain $E_{i,fog}^{prc\&str}$ in Eq. (1):

$$E_{i,fog}^{prc\&str} = L_i . E_{fog}^{prc} + L_i . E_{fog}^{str}, \tag{3}$$

in which E_{fog}^{prc} and E_{fog}^{str} denote the consumed energy for processing and storing each byte of request inside a fog instance, respectively. In the above formula, E_{fog}^{prc} is calculated as follows:

$$E_{fog}^{prc} = \left[\beta_{idle} . P_{idle}^{fog} + (1 - \beta_{idle}) . P_{busy}^{fog} \right] . \frac{\tau^{fog}}{\theta^{fog}}, \tag{4}$$

in which P_{idle}^{fog} and P_{busy}^{fog} denote the idle and busy power consumption of a fog instance, respectively. Also, we denote the ratio of the idle period to the total period for each processor inside fog/cloud by β_{idle}. Simply, the value in brackets gives the average power consumption per data byte. Multiplying this value by the time factor τ^{fog}/θ^{fog} gives us the amount of energy consumption in the processing state in J. Here, τ^{fog} and θ^{fog} denote the time duration (in s) and the size (in byte) of each instruction inside a fog instance, respectively.

We now proceed to calculate $E_{i,f,c}^{fog-cloud}$ in Eq. (1). First, let $E^{fog-cloud}$ denote the consumed energy to offload each byte of data from a fog instance to a cloud datacenter. Now, the consumed energy to offload the request i from a fog instance f to the cloud data center c is obtained as follows:

$$E_{i,f,c}^{fog-cloud} = L_i . E^{fog-cloud}. \tag{5}$$

As mentioned earlier, any IoT device can transmit requests directly to the central cloud. We now explain this operation. In this case, the energy consumption consists of two components: (a) The consumed energy for direct offloading of an object from the IoT instance to a cloud datacenter, $E_{i,c}^{IoT-cloud}$ and (b) The consumed energy for processing and storing the request inside a cloud datacenter, $E_{i,c}^{prc\&str}$. Therefore, the total consumed energy in a cloud datacenter c for each request, E^{cloud}, can be calculated as follows:

$$E_{i,c}^{cloud} = E_{i,c}^{IoT-cloud} + E_{i,c}^{prc\&str} + E_{i,c}^{cloud-IoT}, \quad \forall i \in \mathcal{I} \ \ \exists c \in \mathcal{C}. \tag{6}$$

Similar to what was described in Eq. (1), here $E_{i,c}^{cloud-IoT}$ represents the consumed energy to send the answer from a cloud data center c back to the IoT instance. Similar to Eq. (2), to calculate $E_{i,c}^{IoT-cloud}$ in Eq. (6). We write (Mebrek et al. 2017):

$$E_{i,c}^{IoT-cloud} = L_i . E_{network}^{IoT-cloud} + L_i . h_{IoT-cloud} . \left(\frac{P_{idle}^{cloud}}{C_{idle}^{cloud}} + \frac{P_{\max}^{cloud}}{u^{cloud} . C_{\max}^{cloud}} \right). \tag{7}$$

We now obtain $E_{i,c}^{prc\&str}$ in Eq. (6):

$$E_{i,c}^{prc\&str} = L_i . E_{cloud}^{prc} + L_i . E_{cloud}^{str}, \tag{8}$$

which E_{cloud}^{prc} is calculated as follows:

$$E_{cloud}^{prc} = \left[\beta_{idle} . P_{idle}^{cloud} + (1 - \beta_{idle}) . P_{busy}^{cloud} \right] . \frac{\tau^{cloud}}{\theta^{cloud}}. \tag{9}$$

3.2 Delay

The response time of a request i sent by an IoT object consists of the sum of (a) the transmission latency to offload an object from the IoT instance i to the fog instance f, $t_{i,f}^{IoT-fog}$, (b) the processing latency of the request i inside a fog instance, $t_{i,fog}^{prc}$; (c) in cases that the resources of a fog instance are insufficient to process the request, it offloads the request to the central cloud. We denote this time by $t_{i,f,c}^{fog-cloud}$. Again, we assume that α percent of requests are offloaded from a fog instance to a cloud datacenter. $E_{i,f,c}^{fog-cloud}$. Total response time in the fog instance $f \in \mathcal{F}$ for the request $i \in \mathcal{I}$ is obtained as follows:

$$t_{i,f}^{fog} = t_{i,f}^{IoT-fog} + t_{i,fog}^{prc} + \alpha . t_{i,f,c}^{fog-cloud}, \quad \forall i \in \mathcal{I} \ \ \exists f \in \mathcal{F} \ \ \exists c \in \mathcal{C}. \tag{10}$$

We now proceed to calculate $t_{i,f}^{IoT-fog}$ in Eq. (10). We write:

$$t_{i,f}^{IoT-fog} = L_i . t_{network}^{IoT-fog} + L_i' . t_{network}^{IoT-fog} = (L_i + L_i' .) . t_{network}^{IoT-fog}, \tag{11}$$

in which $t_{network}^{IoT-fog}$ denotes the transmission delay of each byte of request from an IoT instance to the fog instance. Note that Eq. (11) is the sum of two times: the request transmission time and the response transmission time. We now obtain $t_{i,fog}^{prc}$ in Eq. (10):

$$t_{i,fog}^{prc} = L_i . \frac{\tau^{fog}}{\theta^{fog}}. \tag{12}$$

Then, we proceed to obtain $t_{i,f,c}^{fog-cloud}$ in Eq. (10). It includes the sum of (a) the transmission time of the request from the fog instance to cloud datacenter, $L_i . t_{network}^{fog-cloud}$, (b) the delay of processing the request inside the cloud, $L_i . \frac{\tau^{cloud}}{\theta^{cloud}}$; (c) transmission delay of the response of the request, L_i', from the cloud to the IoT instance, $L_i' . t_{network}^{IoT-cloud}$. The total offloading time between the fog instance and the cloud is obtained as follows (Mebrek et al. 2017):

$$t_{i,f,c}^{fog-cloud} = L_i . t_{network}^{fog-cloud} + L_i . \frac{\tau^{cloud}}{\theta^{cloud}} + L_i' . t_{network}^{IoT-cloud}. \tag{13}$$

Equations (10)–(13) describes a situation in which a request from an IoT device is offloaded to a fog instance. As mentioned earlier, any IoT device can also offload requests directly to the central cloud. We now explain this operation. In this case, the delay consists of two components: (a) the transmission latency for direct offloading of an object from the IoT instance i to the cloud datacenter, $t_{i,c}^{IoT-cloud}$ (b) the processing latency of the request inside the cloud datacenter, $t_{i,c}^{prc}$. Therefore, the total delay in a cloud datacenter c for each request can be calculated as follows:

$$t_{i,c}^{cloud} = t_{i,c}^{IoT-cloud} + t_{i,c}^{prc}, \quad \forall i \in \mathcal{I} \quad \exists c \in \mathcal{C}. \tag{14}$$

Similar to Eq. (11), to calculate $t_{i,c}^{IoT-cloud}$ in Eq. (14). We write:

$$t_{i,c}^{IoT-cloud} = L_i . t_{network}^{IoT-cloud} + L_i' . t_{network}^{IoT-cloud} = (L_i + L_i' .) . t_{network}^{IoT-cloud}, \tag{15}$$

which $t_{network}^{IoT-cloud}$ denotes the transmission delay of each byte of request from an IoT instance to the cloud datacenter. Similar to Eq. (12), to calculate $t_{i,c}^{prc}$ in Eq. (14). We write:

$$t_{i,c}^{prc} = L_i . \frac{\tau^{cloud}}{\theta^{cloud}}. \tag{16}$$

3.3 Objective functions and constrains

As mentioned earlier, our goal is to find the best candidates from the fog/cloud nodes to offload any IoT requests to. This operation must be performed in such a way that leads to a trade-off between response time and energy consumption in the system. For this purpose, we define a binary decision variable, $x_{i,f}$. A value of 1 for this variable indicates that a request $i \in \mathcal{I}$ is offloaded to a fog instance $f \in \mathcal{F}$; otherwise, it is zero:

$$x_{i,f} \in \{0,1\}, \quad \forall i \in \mathcal{I} \;\; \forall f \in \mathcal{F}. \tag{17}$$

Similarly, we define a binary decision variable, $x_{i,c}$, for cloud nodes. A value of 1 for this variable indicates that a request $i \in \mathcal{I}$ is offloaded to a cloud datacenter $c \in \mathcal{C}$; otherwise, it is zero:

$$x_{i,c} \in \{0,1\}, \quad \forall i \in \mathcal{I} \;\; \forall c \in \mathcal{C}. \tag{18}$$

Now, according to Eqs. (1) and (6), the power consumption function for an IoT request $i \in \mathrm{I}$ can be defined as follows:

$$E_i = \sum_{f \in \mathrm{F}} E_{i,f}^{fog} . x_{i,f} + \sum_{c \in \mathrm{C}} E_{i,c}^{cloud} . x_{i,c} \quad \forall i \in \mathcal{I}. \tag{19}$$

We now write the aggregate energy consumption of all requests as follows:

$$\tilde{E} = \sum_{i \in \mathrm{I}} E_i. \tag{20}$$

Similarly, according to Eqs. (10) and (14), the response time function for an IoT request $i \in \mathcal{I}$ can be defined as follows:

$$t_i = \sum_{f \in \mathrm{F}} t_{i,f}^{fog} . x_{i,f} + \sum_{c \in \mathrm{C}} t_{i,c}^{cloud} . x_{i,c}, \quad \forall i \in \mathcal{I}, \tag{21}$$

and the aggregate response time of all requests is as follows:

$$\tilde{t} = \sum_{i \in \mathrm{I}} t_i. \tag{22}$$

We now proceed to explain the constraints of the system. Each object can merely be offloaded to one fog/cloud instance. That is,

$$\sum_{f \in \mathrm{F}} x_{i,f} + \sum_{c \in \mathrm{C}} x_{i,c} = 1, \quad \forall i \in \mathcal{I}. \tag{23}$$

The response time of each IoT request $i \in \mathcal{I}$ should not exceed its deadline. That is,

$$t_i \le t_i^{deadline}, \quad \forall i \in \mathcal{I}. \tag{24}$$

The memory allocated to all requests offloaded to a fog instance $f \in \mathcal{F}$ must not exceed its available memory. That is,

$$\sum_{i \in \mathrm{I}} m_i . x_{i,f} \le m_f, \quad \forall f \in \mathcal{F}. \tag{25}$$

Similarly, for each cloud datacenter $c \in \mathcal{C}$, we write

$$\sum_{i \in \mathrm{I}} m_i . x_{i,c} \le m_c, \quad \forall c \in \mathcal{C}. \tag{26}$$

Finally, the optimization problem is formulated as follows:

$$\mathrm{P}_1 : \min_{(\mathbf{x}_{fog}, \mathbf{x}_{cloud})} \{f_1 : \tilde{E} \text{ and } f_2 : \tilde{t}\} \tag{27}$$

$$\text{s.t.Eqs. (23)} - \text{(26)}. \tag{28}$$

3.4 Classic mathematical method

One of the most common ways to solve the above multi-objective problem is to turn it into a single-objective one with the "*weighted sum*" method (Parvizi and Rezvani 2020). Therefore, we rewrite the problem as follows:

$$\mathrm{P}_1 : \min_{(\mathbf{x}_{fog}, \mathbf{x}_{cloud})} F = w_1 . Ln(f_1) + w_2 . Ln(f_2) \tag{29}$$

$$\text{s.t.Eqs. (23)} - \text{(26)} \tag{30}$$

$$w_1 + w_2 = 1 \tag{31}$$

$$0 \le w_1 \le 1 \tag{32}$$

$$0 \le w_2 \le 1.$$

Logarithmic function in Eq. (29) guarantees the existence of Pareto optimality. Pareto optimality is a state in which it is impossible to improve the status of an objective function without degrading another one (Parvizi and Rezvani 2020; Aboutorabi and Rezvani 2020; Tavakoli-Someh and Rezvani 2019). To facilitate problem-solving with convex optimization software, we convert it into a matrix form. We rewrite the fog offloading decision variable in matrix form,$\mathbf{X}_{fog_{N \times F}}$, as follows:

$$\mathbf{X}_{fog} = \begin{bmatrix} x_{1,1} & x_{1,2} & \dots & x_{1,F} \\ x_{2,1} & x_{2,2} & \dots & x_{2,F} \\ . & . & . & . \\ x_{N,1} & x_{N,2} & \dots & x_{N,F} \end{bmatrix}, \quad \forall i \in \mathcal{I} \;\; \forall f \in \mathcal{F}. \tag{33}$$

The i-th row of the matrix $\mathbf{X}_{fog}$ represents the offloading vector corresponding to the i-th IoT request. Therefore, $\mathbf{X}_{fog}^{(1)}$, $\mathbf{X}_{fog}^{(2)}$, …, and $\mathbf{X}_{fog}^{(N)}$ denote the offloading vectors corresponding to first, second, …, and N-th IoT request, respectively. Also, the matrix of cloud offloading decision variables, $\mathbf{X}_{cloud_{N\times M}}$, is as follows:

$$\mathbf{X}_{cloud} = \begin{bmatrix} x_{1,1} & x_{1,2} & \dots & x_{1,M} \\ x_{2,1} & x_{2,2} & \dots & x_{2,M} \\ . & . & . & . \\ x_{N,1} & x_{N,2} & \dots & x_{N,M} \end{bmatrix}, \quad \forall i \in \mathcal{I} \quad \forall c \in \mathcal{C}. \tag{34}$$

Note that in the simulation, we have considered only one cloud datacenter. Therefore, the above decision variable becomes a columnar vector. Similarly, we denote the memory usage and the deadline of IoT requests by matrices $\mathbf{M}_{1\times N}$ and $\mathbf{D}_{1\times N}$, respectively:

$$\mathbf{M} = \begin{bmatrix} m_1 & m_2 & \dots & m_N \end{bmatrix}, \quad \forall i \in \mathcal{I} \tag{35}$$

$$\mathbf{D} = \begin{bmatrix} t_1^{deadline} & t_2^{deadline} & \dots & t_N^{deadline} \end{bmatrix}, \quad \forall i \in \mathcal{I}. \tag{36}$$

The energy consumption, latency, and available memory of fog instances are denoted by matrices $\mathbf{E}'_{N\times F}$, $\mathbf{T}'_{N\times F}$, and $\mathbf{M}'_{1\times F}$, respectively:

$$\mathbf{E}' = \begin{bmatrix} E_{1,1}^{fog} & E_{1,2}^{fog} & \dots & E_{1,F}^{fog} \\ E_{2,1}^{fog} & E_{2,2}^{fog} & \dots & E_{2,F}^{fog} \\ . & . & . & . \\ E_{N,1}^{fog} & E_{N,2}^{fog} & \dots & E_{N,F}^{fog} \end{bmatrix}, \quad \forall i \in \mathcal{I} \quad \forall f \in \mathcal{F} \tag{37}$$

$$\mathbf{T}' = \begin{bmatrix} t_{1,1}^{fog} & t_{1,2}^{fog} & \dots & t_{1,F}^{fog} \\ t_{2,1}^{fog} & t_{2,2}^{fog} & \dots & t_{2,F}^{fog} \\ . & . & . & . \\ t_{N,1}^{fog} & t_{N,2}^{fog} & \dots & t_{N,F}^{fog} \end{bmatrix}, \quad \forall i \in \mathcal{I} \quad \forall f \in \mathcal{F} \tag{38}$$

$$\mathbf{M}' = \begin{bmatrix} m_1 & m_2 & \dots & m_F \end{bmatrix}, \quad \forall f \in \mathcal{F}. \tag{39}$$

Similarly, for cloud datacenters we have

$$\mathbf{E}'' = \begin{bmatrix} E_{1,1}^{cloud} & E_{1,2}^{cloud} & \dots & E_{1,M}^{cloud} \\ E_{2,1}^{cloud} & E_{2,2}^{cloud} & \dots & E_{2,M}^{cloud} \\ . & . & . & . \\ E_{N,1}^{cloud} & E_{N,2}^{cloud} & \dots & E_{N,M}^{cloud} \end{bmatrix}, \quad \forall i \in \mathcal{I} \quad \forall c \in \mathcal{C} \tag{40}$$

$$\mathbf{T}'' = \begin{bmatrix} t_{1,1}^{cloud} & t_{1,2}^{cloud} & \dots & t_{1,M}^{cloud} \\ t_{2,1}^{cloud} & t_{2,2}^{cloud} & \dots & t_{2,M}^{cloud} \\ . & . & . & . \\ t_{N,1}^{cloud} & t_{N,2}^{cloud} & \dots & t_{N,M}^{cloud} \end{bmatrix}, \quad \forall i \in \mathcal{I} \quad \forall c \in \mathcal{C}, \tag{41}$$

and

$$\mathbf{M}'' = \begin{bmatrix} m_1 & m_2 & \dots & m_M \end{bmatrix}, \quad \forall c \in \mathcal{C}. \tag{42}$$

Now, it is possible to formulate the above convex optimization problem in matrix form as follows:

$$P_1 : \min_{\mathbf{X}_{fog},\mathbf{X}_{cloud}} \left\{ w_1 . \sum_{i\in \mathrm{I}} \left(\mathbf{X}_{fog}^{(i)} . \left(\mathbf{E}'^{(i)}\right)^T + \mathbf{X}_{cloud}^{(i)} . \left(\mathbf{E}''^{(i)}\right)^T \right) + w_2 . \sum_{i\in \mathrm{I}} \left(\mathbf{X}_{fog}^{(i)} . \left(\mathbf{T}'^{(i)}\right)^T + \mathbf{X}_{cloud}^{(i)} . \left(\mathbf{T}''^{(i)}\right)^T \right) \right\} \tag{43}$$

$$\text{s.t. } (\mathbf{X}_{fog} + \mathbf{X}_{cloud}) . \mathbf{I}^T = \mathbf{I}^T \tag{44}$$

$$\mathbf{X}_{fog}^{(i)} . \left(\mathbf{T}'^{(i)}\right)^T + \mathbf{X}_{cloud}^{(i)} . \left(\mathbf{T}''^{(i)}\right)^T \leq \mathbf{D} \tag{45}$$

$$\mathbf{M} . \mathbf{X}_{fog} = \mathbf{M}' \tag{46}$$

$$\mathbf{M} . \mathbf{X}_{cloud} = \mathbf{M}''. \tag{47}$$

When the number of IoT requests and fog instances is small, the above problem can be solved by using Non-linear Integer Programming (NIP) methods. We show the pseudocode of this method in Algorithm 1. Unfortunately, as the scale of the problem grows, the time complexity of this algorithm increases dramatically, making it unsuitable for practical applications.

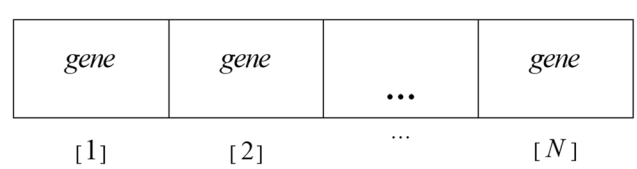

Fig. 2 Structure of a typical "*chromosome*"

Algorithm 1 Pseudo-code of the problem with the ***classic mathematic method***

Input: number of IoT requests N, number of fog instances F, number of cloud datacenters M, energy consumption of fog instances $\mathbf{E}'_{N\times F}$, energy consumption of cloud datacenters $\mathbf{E}''_{N\times M}$, latency of fog instances $\mathbf{T}'_{N\times F}$, latency of cloud datacenters $\mathbf{T}''_{N\times M}$, deadline of IoT requests $\mathbf{D}_{1\times N}$, the memory usage of IoT requests $\mathbf{M}_{1\times N}$, maximum affordable memory of fog instances $\mathbf{M}'_{1\times F}$, maximum affordable memory of cloud datacenters $\mathbf{M}''_{1\times M}$,

Output: fog decision variables $\mathbf{X}^*_{fog}$,+cloud decision variables $\mathbf{X}^*_{cloud}$

1: Calculate $\mathbf{X}^*_{fog}+\mathbf{X}^*_{cloud}$ *as possible Solution of KKT relaxation of Eqs. (43)-(47).*

2: ***If*** $\mathbf{X}^*_{fog}+\mathbf{X}^*_{cloud}$ *is an integer* ***then***

// $\mathbf{X}^*_{fog}+\mathbf{X}^*_{cloud}$ is an optimal solution to the problem.

3: *return* $\mathbf{X}^*_{fog}+\mathbf{X}^*_{cloud}$ *as the optimal solution*

4: ***else***

5: *add an inequality constraint to KKT so that all integer solutions in KKT satisfy, except* $\mathbf{X}^*_{fog}+\mathbf{X}^*_{cloud}$*;*

6: *go to Step #1*

7: ***end if***

4 Problem solution with NSGA-II

As previously explained, one of the most popular methods for solving NP-hard problems, such as offloading, is to use metaheuristic methods. Interested readers are highly recommended to study Keshavarznejad et al. (2021), Parvizi and Rezvani (2020) and Tavakoli-Someh and Rezvani (2019) for implementations of some metaheuristic algorithms. The NSGA-II was proposed by Deb et al. (2002). This method is one of the strongest metaheuristic methods for solving multi-objective problems, such as the case of Eqs. (43)–(47).

Depending on the type of problem, different structures for chromosomes may be suggested (Esfandiari and Rezvani 2021). Figure 2 shows the chromosome structure for the problem of Eqs. (43)–(47). Each index in this array specifies the identifier of an IoT request $i \in \mathcal{I}$, and the value in each cell represents a gene. Note that N represents the total number of IoT requests. Table 2 shows the structure of a gene. Each gene contains the identifier of a candidate fog/cloud instance as well as the energy consumption and the delay of the IoT request i. Each possible solution to the problem is in the form shown in Table 3.

We represent the offspring and parent population in a given time t with $\mathcal{P}_t$ and $\mathcal{O}_t$, respectively. Algorithm 2 shows the *fast_non_dominated_sort* routine. The main task is to extract all Pareto fronts $\mathcal{F}_i$. This procedure takes the population $\mathcal{R}_t$ as input and extracts the number of solutions such as $q \in \mathcal{R}_t$ that dominate a solution $p \in \mathcal{R}_t$. If the dominance of a member on the Pareto front $\mathcal{F}_i$ is zero, it is transferred to a higher Pareto front, $\mathcal{F}_{i+1}$. The lower the Pareto front index, the better the solutions are.

Algorithm 2 Pseudo-code of procedure ***fast_non_dominated_sort*** (Deb et al. 2002)

Input: the union of parents and offspring sets $\mathcal{R}_t$ // $\mathcal{R}_t = \mathcal{P}_t \cup \mathcal{O}_t$

Output: non-dominated Pareto front set $\mathcal{F} = \{\mathcal{F}_1, \mathcal{F}_2, ...\}$

```
for each p ∈ R_t do                    // n_p : Pareto domination counter
    Set n_p = 0
    Set S_p = ∅                        // S_p : The set of solutions that are dominated by a solution p
    for each q ∈ R_t do
        if ( p ≺ q ) then              // if a solution q is dominated by p , add it to S_p
            S_p = S_p ∪ {q}
        else if ( q ≺ p ) then         // else increment the domination counter of p
            n_p = n_p + 1
    end for
    if ( n_p = 0 ) then                // p belongs to the first front
        p_rank = 1
        F_1 = F_1 ∪ {p}
    end if
    i = 1                              // initializing the Pareto front counter
    while F_i ≠ ∅ do
        T = ∅                          // T : A set to store the members of the next front
        for each p ∈ F_i do
            for each q ∈ S_p do
                n_q = n_q − 1
                if ( n_q = 0 ) then    // q belongs to the next non-dominated front
                    q_rank = i + 1
                    T = T ∪ {q}
                end if
            end for
        i = i + 1
        F_i = T
    end while
end for
```

A pseudo-code for *crowding distance* routine is shown in Algorithm 3. It indicates the density of the solutions around a particular solution. It takes a non-dominated set $\mathcal{F}_i$ and returns the value of the distance from the neighboring points after sorting the members of the population based on the value of each objective function m. Here, $\mathcal{F}_i[j].m$ represents the value of m-th objective of the j-th member of the population. Also, x $f_m^{\min}$ and x $f_m^{\max}$ represent the smallest and largest values of the m-th objective function, respectively. Finally, this routine returns the total amount of crowding distance regarding f_1 and f_2.

Algorithm 3 Pseudo-code of procedure ***crowding_distance_assignment*** (Deb et al. 2002)

Input: Pareto frontier $\mathcal{F}_i$, number of objective functions m

Output: crowding distance values $\mathcal{F}_i[1]_{distance} \ldots \mathcal{F}_i[l]_{distance}$

```
Set l = |F_i|                    // number of solutions in F_i
for j = 1 to l do
    Set F_i[j]_distance = 0
end for
for each objective m do
    F_i = sort(F_i, m)           // sort using each objective value
    F_i[1]_distance = F_i[l]_distance = ∞   // boundary points
    for j = 2 to l − 1 do        // for all other points
        F_i[j]_distance = F_i[j]_distance + (F_i[j+1].m − F_i[j−1].m) / (f_m^max − f_m^min)
    end for
end for
```

Table 2 Structure of class "*gene*"

class *gene{*	
int *fog_id;*	// the identifier of candidate fog instance $f \in \mathrm{F}$
int *cloud_id;*	// the identifier of candidate cloud datacenter $c \in \mathrm{C}$
double *E;*	// the energy consumption of the IoT request $i \in \mathrm{I}$ on the candidate fog instance f /cloud datacenter c. This attribute is calculated using Eq. (19)
double *t;*	// the delay of the IoT request $i \in \mathrm{I}$ on the candidate fog instance f /cloud datacenter c. This attribute is calculated using Eq. (21)
}	

Table 3 Structure of class "*solution*"

class *solution{*	
gene *chromosome[N];*	//N denotes the total number of IoT requests
double *E_total;*	// This attribute shows the aggregate energy consumption of all IoT requests which is calculated using Eq. (20). It represents the value of the objective function f_1.
double *t_total;*	// This attribute shows the aggregate delay of all IoT requests which is calculated using Eq. (22). It represents the value of the objective function f_2.
int *Pareto_front_ID;*	// the ID of non-dominated Pareto front for this solution
double *dist;*	// crowding distance of this solution
int *rank;*	// ranking value of this solution
}	

We used the tournament method to *crossover* the chromosomes, the code of which is shown in Algorithm 4. Each of the two parent chromosomes from $\mathcal{P}_t$ is divided into two parts, and then are interchanged and two child chromosomes are created. For each child, fitness is checked. Also, according to Algorithm 5, we select two random points from each child chromosome for the mutation and add the mutated offspring to the current population.

Algorithm 4 Pseudo-code of procedure ***crossover***

Input: selected parent solutions sol_1, sol_2, number of genes in a chromosome N

Output: offspring population O_t

```
divide each of chromosomes sol_1, sol_2 into two segments
Set offspring_1 = Swap two halves of sol_1, sol_2
Set offspring_2 = Swap remaining halves of sol_1, sol_2
for i = 1 to 2 do
    // do the following actions for both offspring_1 and offspring_2
    Set offspring_i = calc_fitness (offspring_i);
    if (offspring_i.E_total <> ∞) && (offspring_i.t_total <> ∞) then    // checking feasibility of
        solutions
        O_t = O_t ∪ {offspring_i}    // add the new solution to the current population
    end if
end for
```

Algorithm 5 Pseudo-code of procedure ***mutation***

Input: An *offspring* which is generated by crossover routine, number of genes in a chromosome N

Output: offspring population O_t

```
Select a random mutation point j
Select another random mutation point k
Swap genes j and k
Set offspring = calc_fitness (offspring);
if (offspring.E_total <> ∞) && (offspring.t_total <> ∞) then    // checking feasibility of mutant
    O_t = O_t ∪ {offspring}    // add the mutant to the current population
end if
```

To calculate the fitness of each solution in Algorithm 6, first, the constraints of Eqs. (44)–(47) are checked. If at least one of these constraints is not met, the chromosome rank is set to a very large number. Thus, the chromosome will no longer have a chance to survive. If the chromosome can meet all the constraints, its rank value is calculated and returned. Chromosome rank is determined by examining the value of the objective function in Eq. (43) relative to other competitors.

Algorithm 6 Pseudo-code of procedure ***calc_fitness***

Input: solution sol , number of genes in a chromosome N

Output: Fitness value f

for $i = 1$ *to* N ***do***

// do the following actions for all genes $i \in \mathbf{I}$ of the chromosome

Set $E_i = sol.\ chromosome[i].E$ using Eq. (19);

Set $t_i = sol.\ chromosome[i].t$ using Eq. (21);

end for

for $i = 1$ *to* N ***do***

// checking if constraints in Eqs. (44)-(47) are violated

if $(\mathbf{X}_{fog} + \mathbf{X}_{cloud}).\mathbf{I}^T \neq \mathbf{I}^T$ OR $\mathbf{X}_{fog}^{(i)} \cdot \left(\mathbf{T}'^{(i)}\right)^T + \mathbf{X}_{cloud}^{(i)} \cdot \left(\mathbf{T}''^{(i)}\right)^T > \mathbf{D}$

OR $(\mathbf{M}.\mathbf{X}_{fog} \neq \mathbf{M}')$ *OR* $(\mathbf{M}.\mathbf{X}_{cloud} \neq \mathbf{M}'')$ ***then***

Set $sol.rank = \infty$;

return $sol.rank$ ***;*** // infeasible offloading solution

end for

Set $sol.E_total = \tilde{E}$ *using* Eq. (43); // feasible offloading solution: calculating the value of objective function

Set $sol.t_total = \tilde{t}$ *using* Eq. (43);

Set $sol.rank =$ ***find_rank***(sol);

return $sol.rank$ ***;***

Algorithm 7 compares the current population with the best non-dominant solutions previously found. Recursively, it finally finds the optimal Pareto front with the desired solutions to the offloading problem. It combines $\mathcal{P}_t$ and $\mathcal{O}_t$ to create a new set $\mathcal{R}_t$ of size N. After searching among the solutions in the best non-dominated set $\mathcal{F}_1$, it finally adds the members in the solutions $\mathcal{F}_2, \mathcal{F}_3, \ldots, \mathcal{F}_l$ to $\mathcal{P}_{t+1}$. This process is repeated until the size of the solution reaches N.

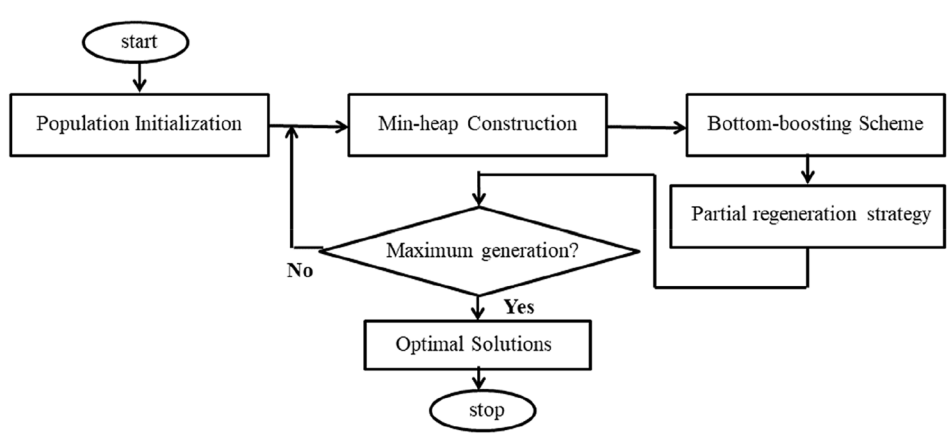

Fig. 3 Flowchart of minimax differential evolution (MMDE) (Kaur et al. 2020a)

Algorithm 7 pseudo-code of procedure ***NSGA-II***

Input: parent population $\mathcal{P}_t$, offspring population $\mathcal{O}_t$, number of genes in a chromosome N

Output: void

$\mathcal{R}_t = \mathcal{P}_t \cup \mathcal{O}_t$

$\mathcal{F} = \textit{fast_non_dominated_sort}(\mathcal{R}_t)$

$\mathcal{P}_{t+1} = \varnothing$

$i = 1$

until $|\mathcal{P}_{t+1}| + |\mathcal{F}_i| \le N$ ***do*** // until the parent population is filled

  crowding_distance_assignment$(\mathcal{F}_i)$

  $\mathcal{P}_{t+1} = \mathcal{P}_{t+1} \cup \mathcal{F}_i$ // $\mathcal{F}_i$: i th non-dominated front

  $i = i + 1$ // inspecting the next front

end until

sort$(\mathcal{F}_i, \prec_n)$ // sort in descending order

$\mathcal{P}_{t+1} = \mathcal{P}_{t+1} \cup \mathcal{F}_i[1:(N - |\mathcal{P}_{t+1}|)]$ // selecting the first $(N - |\mathcal{P}_{t+1}|)]$ individuals in $\mathcal{F}_i$

$\mathcal{O}_{t+1} =$ ***make_new_pop***$(\mathcal{P}_{t+1})$

if *(number of iterations not reached)* ***then***

  NSGA_II$(\mathcal{P}_{t+1}, \mathcal{O}_{t+1})$

end if

Almost a decade after the invention of NSGA-II, another algorithm called NSGA-III was introduced (Parvizi and Rezvani 2020). It starts with a random population of size N and a set of widely distributed H-dimensional *reference points*. NSGA-III uses the reference points to maintain diversity among solutions, while NSGA-II uses a more consistent design through its crowding distance operator for this purpose. The main advantage of the NSGA-III lies in the use of reference points. Although this feature is especially useful in optimization problems that have many objective functions, it may become an Achilles heel in single-objective and double-objective optimization problems. In such cases, one must be very careful in choosing the reference points. Interested readers can refer to Seada and Deb (2015) to study more on this.

In recent years, some complementary heuristics have been proposed to increase the efficiency of metaheuristic methods. Qiu et al. (2017) developed a method called minimax differential evolution (MMDE), which can find optimal solutions, even for asymmetric problems with a fewer number of iterations. Simply speaking, this method in combination with the NSGA-II, moves the solutions towards better areas. Another advantage of this method is that it does not waste time examining the worst-case scenarios. Therefore, it may also increase the rate of convergence. The MMDE solves this problem by using a technique called *bottom-boosting* (Kaur et al. 2020a). Using a new mutation operator and a *partial-regeneration strategy*, the MMDE does not allow the spread of weak solutions. Figure 3 shows the flowchart of this method, which includes five steps:

Step 1. Population Initialization: Each individual in the initial population is a pair of *scenarios* and *solutions*.
Step 2. Min-heap Construction: Each individual is placed as a node of min-heap. The objective function is also considered as the node's key. This will insert a node whose key is the smallest at the root. Due to the low computational complexity, the min-heap structure is practically efficient.
Step 3. Bottom-boosting Scheme: It is used to update node scenarios and evaluate the fitness of the objective function. Frequent removal of root nodes from the min-heap leads to the production of a non-descending population of individuals based on the value of each objective function. An experimental scenario is formed using the "*DE/rand/1*" mutation and *binomial recombination* (Tušar and Filipič 2007). The trail scenario is then tested by the original solution and the new value of the objective function is calculated. If the new value is better than the previous one, the scenario of the original individual will be updated, otherwise, the previous scenario will be resumed.

Step 4. Partial-regeneration Strategy: At this moment, mutant solutions are generated using the DE/rand/1 mutation process. Then, binomial recombination is performed to produce the trial solution using its original and mutated solutions. Finally, the objective value of the trial solution is tested against the original solution, and whichever produces a better fit is selected for the next generation.
Step 5. Termination: The optimization process ends when the maximum number of generations is reached.

The process of generating the optimal response using the MMDE is shown in Algorithm 8. Initially, the required parameters such as population size N_s, maximum generation M_g, scenario dimension S_d, and solution dimension Sol_D are set. Each individual $(Y_{j,a}, C_{j,a})$ is represented as a pair of *solutions* and *scenarios*. Then, each individual's fitness value is calculated using Algorithm 9. Once a min-heap is generated from the current population P_a using Algorithm 10, a bottom-boosting strategy is applied to update the scenarios. Now, after removing the root node, the current population is updated and individuals are sorted in descending order based on their fitness value. The first individual from the updated population is selected as the best solution ever. The current population is then updated using Algorithm 11. The next generation is continued by the algorithm until the termination condition is met. The objective functions in Eqs. (43)–(47) are used to evaluate the fitness of the generated solutions using Algorithm 9.

Algorithm 8 pseudo-code of procedure ***MMDE*** (Kaur et al. 2020a)

Input: Population size N_s, maximum generation M_g, scenario dimension S_d, solution dimension Sol_D

Output: Optimal population

Set generation $a = 0$

Initialize the population randomly such as $P_a = \{(Y_{1,a}, C_{1,a}), (Y_{2,a}, C_{2,a}), \ldots, (Y_{N_s,a}, C_{N_s,a})\}$, in which $Y_{j,a} = \{y^1_{j,a}, y^2_{j,a}, \ldots, y^{Sol_D}_{j,a}\}$ and $C_{j,a} = \{c^1_{j,a}, c^2_{j,a}, \ldots, c^{S_d}_{j,a}\}$, $j = 1, 2, \ldots, N_s$;

for $b = 1$ *to* N_s ***do***

  Calculate the fitness of individual $(Y_{j,a}, C_{j,a})$ using ***Algorithm 9***;

end

for $a = 1$ *to* M_g ***do***

  Generate a min-heap by considering every individual in P_a as a node and its corresponding fitness value as a key;

  Apply bottom-boosting strategy on obtained min-heap using ***Algorithm 10***;

  for $j = 1$ *to* N_s ***do***

    //Individual (Y_r, C_r) stored as a root node in the min-heap

    $Y_{j,a} = Y_r$

    $C_{j,a} = C_r$

    Remove root node from min-heap;

  end

  $Y_{best} = Y_{1,a}$

  Apply partial-regeneration scheme for updating of P_a *using* ***Algorithm 11***;

  $P_{a+1} = P_a$

end

Algorithm 9 pseudo-code of procedure ***fitness_MMDE***

Input: solution $Y_j\,(j=1:Sol_D)$, number of genes in a chromosome N,

Output: Fitness value f

Set $sol = Y_j$

for $i = 1$ *to* N ***do***

// do the following actions for all genes $i \in \mathbf{I}$ of the chromosome

Set $E_i = sol.chromosome[i].E$ using Eq. (19);

Set $t_i = sol.chromosome[i].t$ using Eq. (21);

end for

for $i = 1$ *to* N ***do***

// checking if constraints in Eqs. (44)-(47) are violated

if $(\mathbf{X}_{fog} + \mathbf{X}_{cloud}).\mathbf{I}^T \neq \mathbf{I}^T$ OR $\mathbf{X}_{fog}^{(i)} \cdot \left(\mathbf{T}'^{(i)}\right)^T + \mathbf{X}_{cloud}^{(i)} \cdot \left(\mathbf{T}''^{(i)}\right)^T > \mathbf{D}$

OR $(\mathbf{M}.\mathbf{X}_{fog} \neq \mathbf{M}')$ *OR* $(\mathbf{M}.\mathbf{X}_{cloud} \neq \mathbf{M}'')$ ***then***

Set $sol.rank = \infty$;

return $sol.rank$ ***;*** // infeasible offloading solution

end for

Set $sol.E_total = \tilde{E}$ *using* Eq. (43); // feasible offloading solution: calculating the value of objective function

Set $sol.t_total = \tilde{t}$ *using* Eq. (43);

Set $sol.rank =$ ***find_rank***(sol);

return $sol.rank$ ***;***

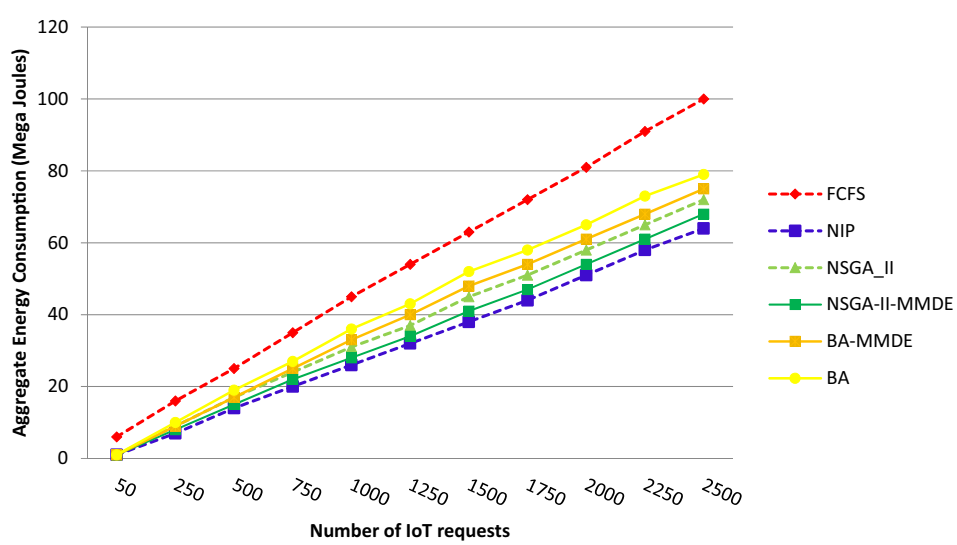

Fig. 4 Comparison of aggregate energy consumption for different numbers of IoT requests

Algorithm 10 pseudo-code of procedure ***Bottom-boosting Strategy*** (Kaur et al. 2020a)

Input: Total number of fitness function evaluations F_v, a min-heap generated from P_a, scaling factor F, and crossover rate C_r

Output: Updated min-heap

Set $ff = 0$
while $ff < F_v$ ***do***
// Assume that the individual stored in the root node is $(Y_{j,a}, C_{j,a})$
Read the root node of the min-heap;
Produce mutant scenario as $CV_{j,a} = C_{r_1,a} + F(C_{r_2,a} - C_{r_3,a})$ //in which r_1, r_2, and r_3 are randomly selected indices from $\{1, 2, ..., N_s\}$
Obtain the trial scenario $CU_{j,a}$ by applying binomial recombination on $C_{j,a}$ and $CV_{j,a}$;
if $f(Y_{j,a}, CU_{j,a}) > f(Y_{j,a}, C_{j,a})$ ***then***
Remove the root node from the min-heap;
$C_{j,a} = CU_{j,a}$
Remove root node from min-heap;
Insert the update $(Y_{j,a}, C_{j,a})$ into the min-heap;
end if
$ff = ff + 1$
end while

Algorithm 11 pseudo-code of procedure ***Partial-regeneration Scheme*** (Kaur et al. 2020a)

Input: Number of regenerated individuals β, sorted population P_a w.r.t their fitness values in non-descending order, crossover rate C_r, and scaling factor F

Output: Updated population P_a

Set $b = 1$
while $b \le \beta$ ***do***
$W_{j,a} = Y_{j,a} + F(Y_{r_1,a} - Y_{r_2,a})$; // Here, the index j is selected from $\{1, 2, ..., \beta\}$ randomly
Obtain the trail solution $\varphi_{j,a}$ using binomial recombination on $Y_{j,a}$ and $W_{j,a}$;
$Y_{N_s+1-b,a} = \varphi_{j,a}$;
Reinitialize $C_{N_s+1-b,a}$ randomly;
Calculate the fitness of updated individual $(Y_{N_s+1-b,a}, C_{N_s+1-b,a})$ using ***Algorithm 9***;
$b = b + 1$
end while

Table 4 Specifications of system structure for simulation

Module	MIPS	RAM	UP bandwidth	Down bandwidth	Level	Rate per MIPS	Busy power	Idle power
Cloud	44,800	40,000	10,000	10,000	0	0.1	16*103	16*83.25
Gateway (fog)	2800	4000	10,000	10,000	1	0.0	107.339	83.433

Table 5 The ANOVA test for aggregate energy consumption

	Sum of squares	df	Mean square	F	Sig.
Between groups	3120.667	5	624.133	1.016	0.417
Within groups	36,876.364	60	614.606		
Total	39,997.030	65			

5 Performance evaluation

5.1 Experimental setting

We used the iFogSim tool (Gupta et al. 2017) to simulate the proposed method. Also, parts of the analysis were performed with SPSS. The tests were performed on a 64-bit Intel® Core™ i5-8269U processor with 6 MB of cache, 4 cores, 4.20 GHz processor frequency, and 8 GB of RAM.

Table 6 LSD pairwise comparisons concerning the aggregate energy consumption

(I) Approach	(J) Approach	Mean difference (I–J)	Std. error	Sig.	95% Confidence interval	
					Lower bound	Upper bound
FCFS	NIP	21.181818*	10.571024	0.050	0.03662	42.32702
	NSGA-II	16.181818	10.571024	0.131	– 4.96338	37.32702
	NSGA-II-MMDE	19.000000	10.571024	0.077	– 2.14520	40.14520
	BA	11.363636	10.571024	0.287	– 9.78156	32.50883
	BA-MMDE	14.272727	10.571024	0.182	– 6.87247	35.41792
NIP	FCFS	– 21.181818*	10.571024	0.050	– 42.32702	– 0.03662
	NSGA-II	– 5.000000	10.571024	0.638	– 26.14520	16.14520
	NSGA-II-MMDE	– 2.181818	10.571024	0.837	– 23.32702	18.96338
	BA	– 9.818182	10.571024	0.357	– 30.96338	11.32702
	BA-MMDE	– 6.909091	10.571024	0.516	– 28.05429	14.23611
NSGA-II	FCFS	– 16.181818	10.571024	0.131	– 37.32702	4.96338
	NIP	5.000000	10.571024	0.638	– 16.14520	26.14520
	NSGA-II-MMDE	2.818182	10.571024	0.791	– 18.32702	23.96338
	BA	– 4.818182	10.571024	0.650	– 25.96338	16.32702
	BA-MMDE	– 1.909091	10.571024	0.857	– 23.05429	19.23611
NSGA-II-MMDE	FCFS	– 19.000000	10.571024	0.077	– 40.14520	2.14520
	NIP	2.181818	10.571024	0.837	– 18.96338	23.32702
	NSGA-II	– 2.818182	10.571024	0.791	– 23.96338	18.32702
	BA	– 7.636364	10.571024	0.473	– 28.78156	13.50883
	BA-MMDE	– 4.727273	10.571024	0.656	– 25.87247	16.41792
BA	FCFS	– 11.363636	10.571024	0.287	– 32.50883	9.78156
	NIP	9.818182	10.571024	0.357	– 11.32702	30.96338
	NSGA-II	4.818182	10.571024	0.650	– 16.32702	25.96338
	NSGA-II-MMDE	7.636364	10.571024	0.473	– 13.50883	28.78156
	BA-MMDE	2.909091	10.571024	0.784	– 18.23611	24.05429
BA-MMDE	FCFS	– 14.272727	10.571024	0.182	– 35.41792	6.87247
	NIP	6.909091	10.571024	0.516	– 14.23611	28.05429
	NSGA-II	1.909091	10.571024	0.857	– 19.23611	23.05429
	NSGA-II-MMDE	4.727273	10.571024	0.656	– 16.41792	25.87247
	BA	– 2.909091	10.571024	0.784	– 24.05429	18.23611

*The mean difference is significant at the 0.05 level

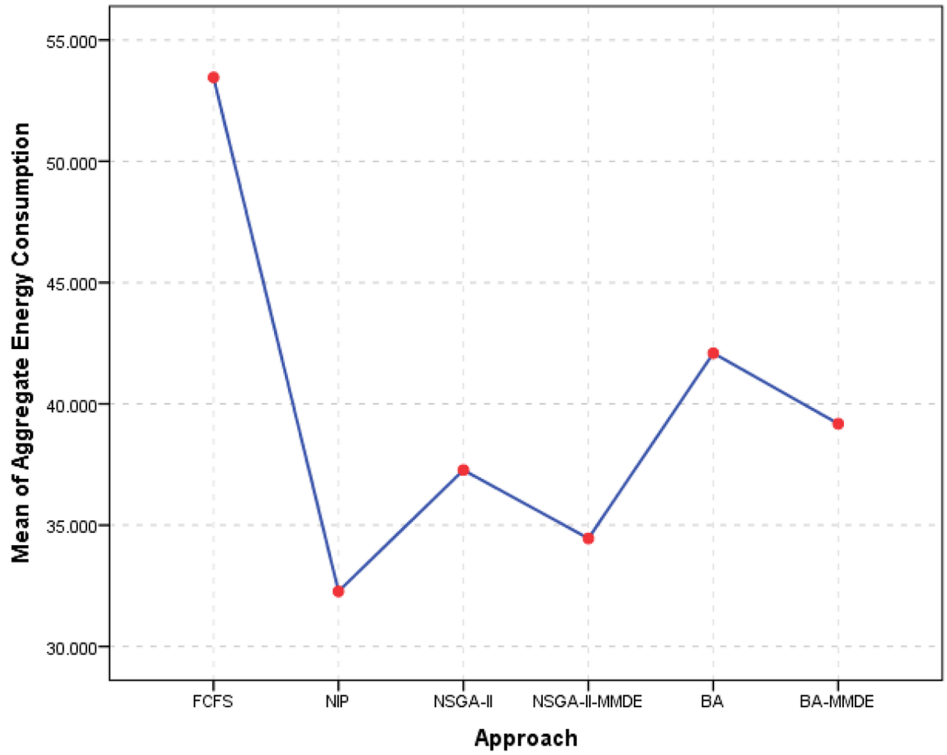

Fig. 5 The distance between the means of aggregate energy consumption resulted from LSD analysis

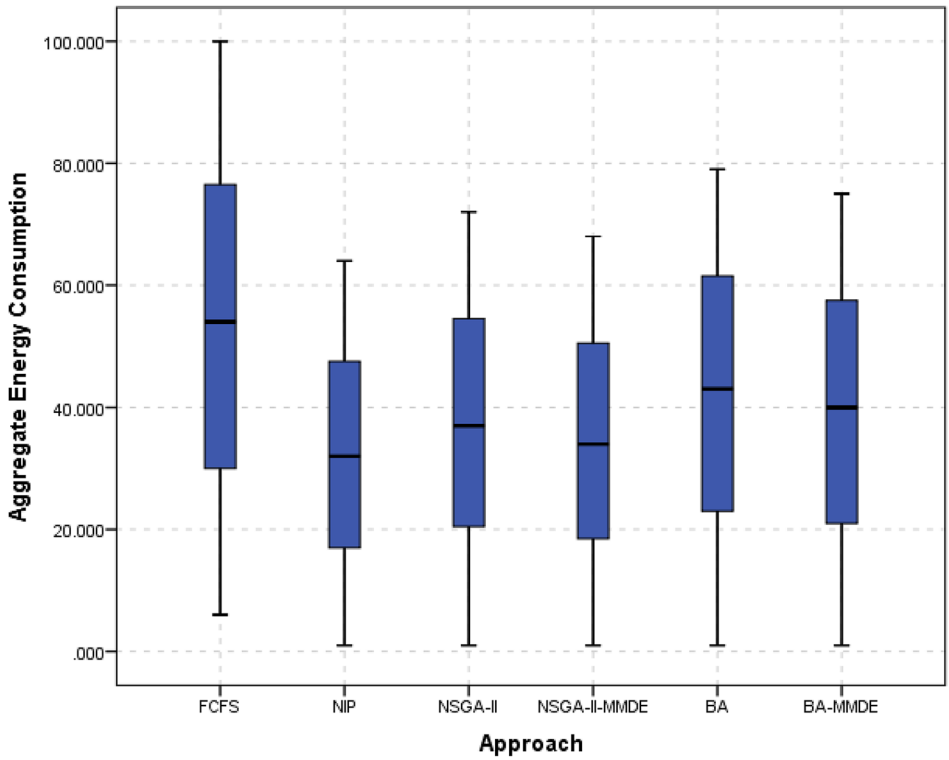

Fig. 6 Comparison of mean and variance of different approaches regarding aggregate energy consumption

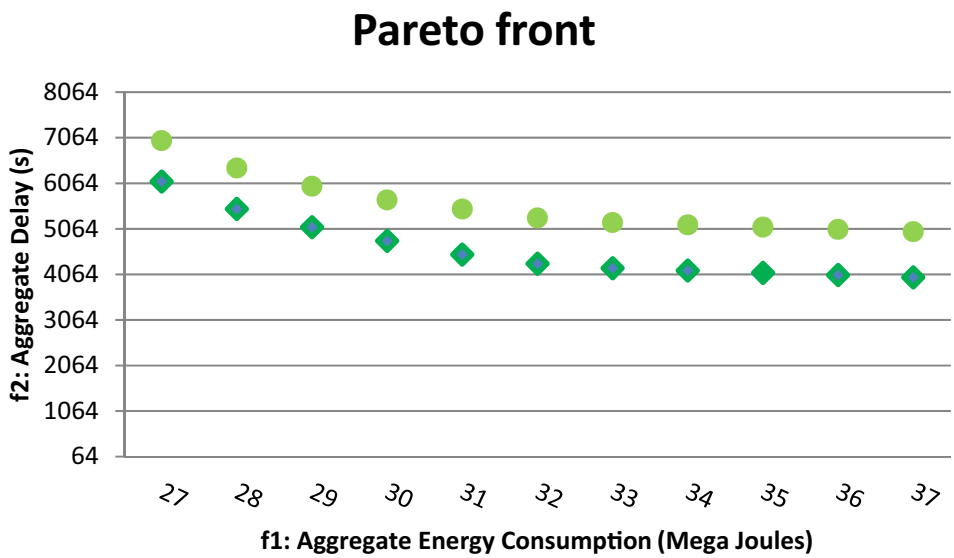

Fig. 7 The Pareto frontier of the NSGA-II (green) and NSGA-II-MMDE (dark green) approaches for 1000 IoT requests (color figure online)

Table 7 Optimal values of the NSGA-II approach for 1000 IoT requests regarding different weights

Solution	w_1	w_2	f_1	f_2
1	1	0	27	7000
2	0.9	0.1	28	6400
3	0.8	0.2	29	6000
4	0.7	0.3	30	5700
5	0.6	0.4	31	5500
6	0.5	0.5	32	5300
7	0.4	0.6	33	5200
8	0.3	0.7	34	5150
9	0.2	0.8	35	5100
10	0.1	0.9	36	5050
11	0	1	37	5000

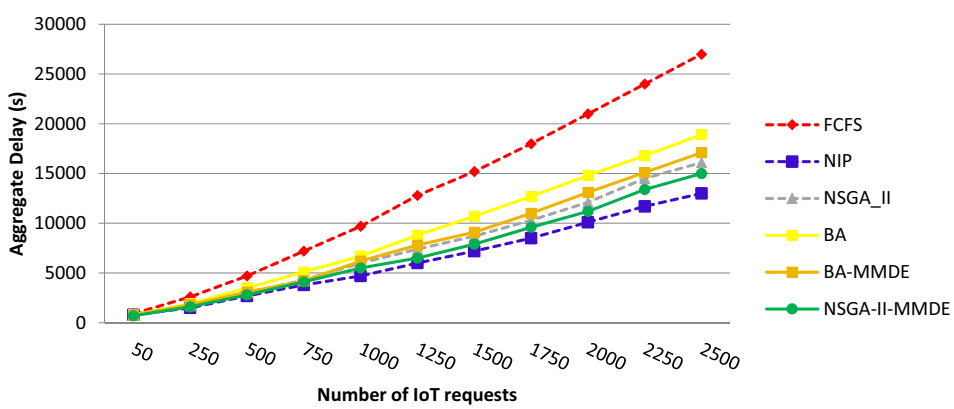

Fig. 8 Comparison of aggregate delay for different numbers of IoT requests

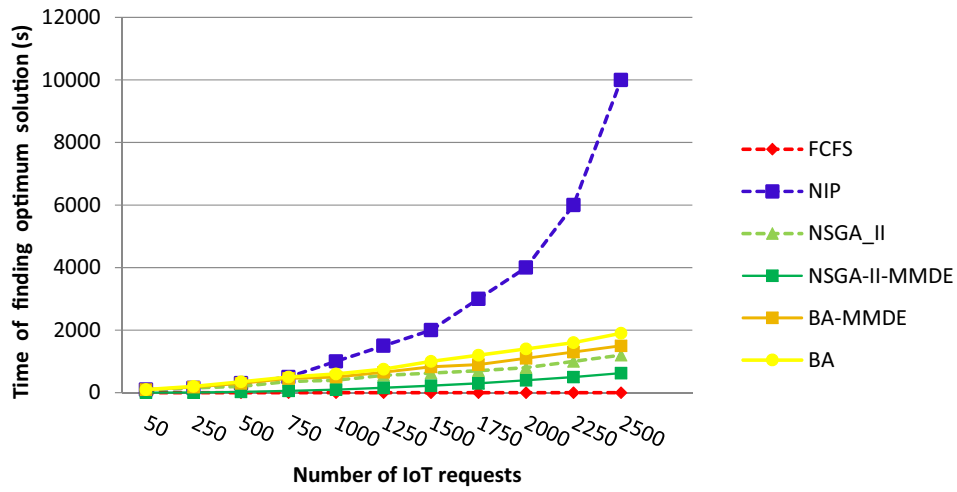

Fig. 9 Comparison of convergence time for different numbers of IoT requests

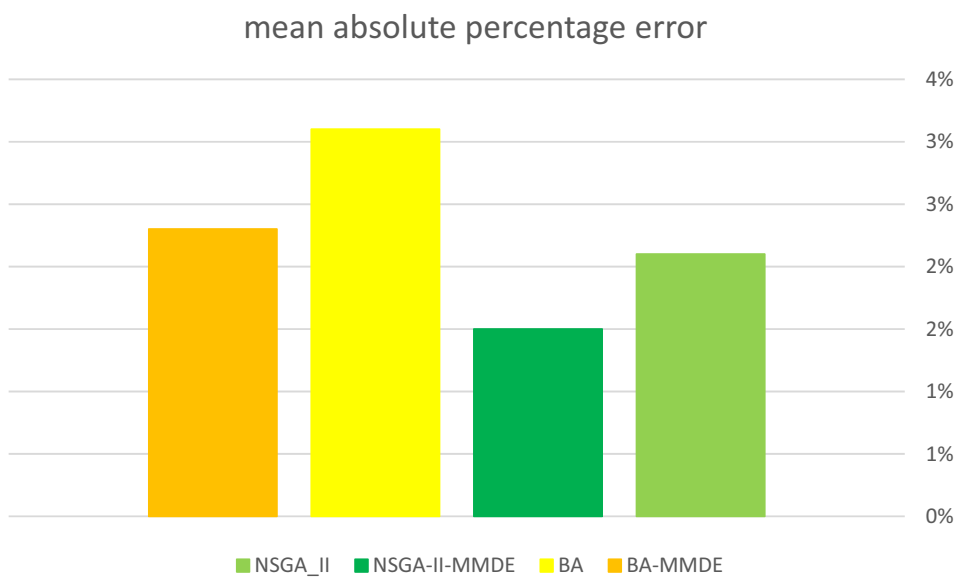

Fig. 10 Comparison of mean absolute percentage error of different metaheuristic approaches

Also, the deadline for each IoT request is a random number with a uniform distribution in the interval [500_{ms}–50_{s}]. The specifications of the system structure are shown in Table 4.

Each fog node as well as each IoT device has a queue (buffer). This queue buffers incoming requests and offloads them to a higher level. The default settings of iFogSim use the FCFS policy, which means that requests are either served or offloaded to the cloud data center as they arrive. We compare FCFS results with both deterministic and random methods. As a deterministic method, we use the NIP based on Algorithm 1 (Eqs. (43)–(47)). We compare NIP against NSGA-II and Bee algorithm. The pseudo-codes of NSGA-II were previously presented in Algorithms 2–7. To evaluate more accurately, we designed other variants of these two approaches based on Algorithms 8–11. The combination of the MMDE approach with NSGA-II and Bees algorithm is abbreviated with *NSGA-II-MMDE* and *BA-MMDE*, respectively. Comparisons have been made in terms of convergence time, run time, and energy consumption criteria. For each fog node, we set the queue size to 20 tasks. Also, all experiments were repeated 40 times. In all curves of this section, average values are used.

We compare the solution obtained from each method with that of the global optimal solution (NIP) in Algorithm 1. For this purpose, we use the mean absolute error (MAE) criterion (Kaur et al. 2020a; Qiu et al. 2017). This value is equal to the difference between the global solution and the best solution obtained from each method as follows:

$$MAE(X_{best}, X_{opt}) = \sum_{j=1}^{Sol_D} \left| x_{best}^{j} - x_{opt}^{j} \right|, \tag{48}$$

which X_{best} is the best solution found by the algorithm, X_{opt} is the optimal global solution, sol_D is the dimensions of the solution space, and x^j_{best} and x^j_{opt} are the variables in dimension j. As previously stated in Eqs. (43)–(47), this study uses two objective functions.

5.2 Analysis of results

Figure 4 shows the comparison of total energy consumption for different IoT requests. Here, the aggregate energy consumption is calculated using Eq. (20). As can be seen from the figure, the FCFS method has the highest total energy consumption because it offloads IoT requests without taking into account the optimality conditions and only based on the deadline. It is also observed that the energy consumption of the NSGA-II-MMDE method is very close to that of the classic mathematical method (NIP).

We use analysis of variance (ANOVA) to determine the difference between the mean of the three methods FCFS, NIP, and NSGA-II. Hypothesis H_0 and H_1 are defined as follows:

H_0: The average energy consumption of all methods is equal.

H_1: At least, there is one method whose average energy consumption is significantly different from the other methods.

Table 5 shows the between-group and within-group statistics of the ANOVA test. By default, we assume a 95% confidence interval. Since the value of the significance ($Sig. = 0.417$) is greater than 0.05, the hypothesis H_1 is accepted.

The ANOVA test determines whether there is a difference between the energy consumption of the methods, but does not provide any information on the mean values of each method. To find out this, some sort of Post Hoc analyses must be used. We run the Fisher's least significant difference (LSD) test (Fisher 2002). Table 6 shows the results of pairwise comparisons. Also, Fig. 5 shows the distance between the average energy consumption of different methods. As can be seen in the figure, the average energy consumption of the NSGA-II-MMDE method is not much different from that of the NIP method. However, the average energy consumption of the FCFS method is significantly higher than that of other methods.

Figure 6 shows the variance of different methods. As can be seen, because the NIP is a deterministic method, its variance is always constant and the lowest. After that, the NSGA-II-MMDE and NSGA-II methods have the lowest variance among all. It can be concluded that the variance of the NSGA-based methods, in any case, is less than that of the BA.

Figure 7 shows the Pareto frontier diagram of the two objective functions f_1 and f_2 for the NSGA-II and the NSGA-II-MMDE approaches. Recall from Eq. (27) that f_1 and f_2 represent the total energy consumption, and the total delay of IoT requests, respectively. To produce Fig. 7, we used the weights in Table 7 and added them to Eqs. (43)–(47). For example, we have shown the optimal values of the NSGA-II approach for 1000 IoT requests with different weights. As shown in Fig. 7, both methods succeed in finding the optimal Pareto points per weight. It is clear that when most IoT requests are processed in local nodes, the overall execution delay is reduced. However, mobile devices will require more battery charge than ever before.

Figure 8 shows the comparison of aggregate delay for different numbers of IoT requests. It is the delay from the moment the task is submitted to the fog/cloud instance until the task is completed. This value is calculated using Eq. (22). As can be seen from the figure, the response time of the NSGA-II-MMDE method is almost the same as that of the NIP method.

Before offloading tasks to the relevant fog/cloud instances are done, the optimal offloading assignments must be calculated. For the NIP method, these calculations are performed using Algorithm 1. For the NSGA-II method, the optimal offloading assignments are performed using Algorithms 2–7. Also, for the NSGA-II-MMDE and the BA-MMDE methods, the optimal calculations are performed using Algorithms 8–11. Figure 9 shows the convergence time of different methods. As mentioned earlier, finding the optimal offloading assignments falls into the category of NP-hard knapsack problems (Keshavarznejad et al. 2021). Therefore, when the number of IoT requests becomes too large, the NIP method requires a lot of time to converge. In contrast, other methods can find a near-optimal solution in much less time. Again, the NSGA-II-MMDE has a faster convergence rate than other metaheuristic methods.

Finally, we calculate the accuracy of the various metaheuristic methods using the mean absolute percentage error criterion. These calculations are based on Eq. (48) which we stated before in this section. As can be seen in Fig. 10, the error ratio of NSGA-II-MMDE is the least among all other methods. Conversely, the BA has the lowest accuracy of all methods.

6 Conclusion and future trends

In this paper, we targeted one of the most significant issues in IoT-fog-cloud environments, namely computation offloading of IoT tasks. We formulated the problem of joint optimization of energy consumption and latency. This modeling does not require any specific assumptions about task input traffic. It takes into account both the cost of the decision and

the cost of offloading/execution. For each task, the output of the decision step specifies which fog/cloud device each task should be sent to. Given that the decision sub-problem falls into the category of NP-hard problems, we solved it using two state-of-the-art metaheuristic methods, namely NSGA-II and Bees algorithm. Also, to improve the quality of the obtained solutions, we combined each of these methods with a robust type of differential evolution method, called the MMDE.

Our results revealed that the NSGA-II-MMDE and the NSGA-II methods have the best performance in terms of significant criteria such as energy consumption, time delay, and mean absolute error. The efficiency of BA-based methods was significantly lower than the efficiency of NSGA-based ones. They could not compete with NSGA-based methods even in combination with MMDE. The average energy consumption of NSGA-II-MMDE and NSGA-II is approximately 34 and 37 kilojoules (kJ), respectively, while for BA-based methods it cannot be less than 39 kJ. Also, the variance of energy consumption for NSGA-based methods is less than BA-based ones, which indicates their greater stability. Also, for a large number of IoT requests, the execution time of NSGA-based methods is approximately 2 s less than BA-based methods. Also, experiments showed that all metaheuristic methods except the NIP method succeed in finding the optimal solution in less than 2 s. Due to the NP-hard nature of the offloading problem, the NIP method is practically inefficient when the number of IoT requests is high.

Several suggestions can be outlined for pursuing this line of research in the future. This research can be done taking into account the type of IoT requests. The use of reinforcement learning, as well as clustering methods in combination with metaheuristic methods, are also interesting topics for future research. Also, combining MMDE methods with other evolutionary optimization techniques such as hyperchaotic maps can be used in critical areas such as medical image processing. One of these areas is the extraction of features to multimodal medical image fusion, which is inherently an NP-hard problem (Kaur and Singh 2021). Another area of application is the use of hyperchaotic maps to generate secure keys in color image encryption. This operation suffers from hyper-tuning problems due to the requirements of the initial parameters. It is possible to use a combination of NSGA-based methods and local chaotic search for color image encryption and tune the 5D chaotic map hyperparameters (Kaur et al. 2020b).

Funding The authors did not receive support from any organization for the submitted work.

Data availability The datasets generated during and analyzed during the current study are available in the [MENDELEY] repository, http://data.mendeley.com/datasets/m6v5kjp9fd/1.

Declarations

Conflict of interest On behalf of all authors, the corresponding author states that there is no conflict of interest.

References

Abbasi M, Pasand EM, Khosravi MR (2020) Workload allocation in IoT-fog-cloud architecture using a multi-objective genetic algorithm. J Grid Comput 18:43–56

Aboutorabi SJS, Rezvani MH (2020) An optimized meta-heuristic bees algorithm for players' frame rate allocation problem in cloud gaming environments. Comput Games J 9(3):281–304

Adhikari M, Gianey H (2019) Energy efficient offloading strategy in fog-cloud environment for IoT applications. Internet Things 6:100053

Adhikari M, Srirama SN, Amgoth T (2019) Application offloading strategy for hierarchical fog environment through swarm optimization. IEEE Internet Things J 7(5):4317–4328

Besharati R, Rezvani MH (2019) A prototype auction-based mechanism for computation offloading in fog-cloud environments. In: 2019 5th conference on knowledge based engineering and innovation (KBEI). IEEE, Tehran, Iran, pp 542–547

Buyya R, Srirama SN (eds) (2019) Fog and edge computing: principles and paradigms. Wiley, Hoboken

Caiza G, Saeteros M, Oñate W et al (2020) Fog computing at industrial level, architecture, latency, energy, and security: a review. Heliyon 6(4):e03706

Chiti F, Fantacci R, Picano B (2018) A matching theory framework for tasks offloading in fog computing for IoT systems. IEEE Internet Things J 5(6):5089–5096

De Maio V, Kimovski D (2020) Multi-objective scheduling of extreme data scientific workflows in Fog. Future Gener Comput Syst 106:171–184

Deb K, Agrawal S, Pratap A et al (2002) A fast and elitist multiobjective genetic algorithm: NSGA-II. IEEE Trans Evol Comput 6(2):182–197

Dinh THL, Kaneko M, Fukuda EH et al (2021) Energy efficient resource allocation optimization in fog radio access networks with outdated channel knowledge. IEEE Trans Green Commun Netw 5(1):146–159

Djemai T, Stolf P, Monteil T et al (2019) A discrete particle swarm optimization approach for energy-efficient IoT services placement over fog infrastructures. In: 2019 18th international symposium on parallel and distributed computing (ISPDC). IEEE, Amsterdam, Netherlands, pp 32–40

Elashri S, Azim A (2020) Energy-efficient offloading of real-time tasks using cloud computing. Cluster Comput 23:3273–3288

Esfandiari S, Rezvani MH (2021) An optimized content delivery approach based on demand–supply theory in disruption-tolerant networks. Telecommun Syst 76:265–289

Fisher GG (2002) Work/personal life balance: a construct development study. Diss Abstr Int Sect B: Sci Eng 63(1-B):575

Ghobaei-Arani M, Souri A, Safara F et al (2020) An efficient task scheduling approach using moth-flame optimization algorithm for cyber-physical system applications in fog computing. Trans Emerg Telecommun Technol 31(2):e3770

Gupta H, Vahid Dastjerdi A, Ghosh SK et al (2017) iFogSim: a toolkit for modeling and simulation of resource management techniques

in the Internet of Things, Edge and Fog computing environments. Softw Pract 47:1275–1296
Huang X, Yang Y, Wu X (2019) A meta-heuristic computation offloading strategy for IoT applications in an edge-cloud framework. In: Proceedings of the 2019 3rd international symposium on computer science and intelligent control, Amsterdam, Netherlands, pp 1–6
Hussein MK, Mousa MH (2020) Efficient task offloading for IoT-based applications in fog computing using ant colony optimization. IEEE Access 8:37191–37201
Jazayeri F, Shahidinejad A, Ghobaei-Arani M (2021) Autonomous computation offloading and auto-scaling the in the mobile fog computing: a deep reinforcement learning-based approach. J Ambient Intell Human Comput 12:8265–8284
Jiang YL, Chen YS, Yang SW, Wu CH (2019) Energy-efficient task offloading for time-sensitive applications in fog computing. IEEE Syst J 13(3):2930–2941
Kaur M, Kumar V (2018) Parallel non-dominated sorting genetic algorithm-II-based image encryption technique. Imaging Sci J 66(8):453–462
Kaur M, Singh D (2021) Multi-modality medical image fusion technique using multi-objective differential evolution based deep neural networks. J Ambient Intell Humaniz Comput 12(2):2483–2493
Kaur M, Singh D, Kumar V (2020a) Color image encryption using minimax differential evolution-based 7D hyper-chaotic map. Appl Phys B 126(9):1–19
Kaur M, Singh D, Sun K, Rawat U (2020b) Color image encryption using non-dominated sorting genetic algorithm with local chaotic search based 5D chaotic map. Future Gener Comput Syst 107:333–350
Keshavarznejad M, Rezvani MH, Adabi S (2021) Delay-aware optimization of energy consumption for task offloading in fog environments using metaheuristic algorithms. Cluster Comput. https://doi.org/10.1007/s10586-020-03230-y
Lahmar IB, Boukadi K (2020) Resource allocation in fog computing: a systematic mapping study. In: 2020 fifth international conference on fog and mobile edge computing (FMEC). IEEE, Paris, France, pp 86–93. https://doi.org/10.1109/FMEC49853.2020.9144705
Liu L, Chang Z, Guo X et al (2018) Multiobjective optimization for computation offloading in fog computing. IEEE Internet Things J 5(1):283–294
Maity S, Mistry S (2020) Partial offloading for fog computing using P2P based file-sharing protocol. In: Das H, Pattnaik P, Rautaray S, Li KC (eds) Progress in computing, analytics and networking. Advances in intelligent systems and computing, vol 1119. Springer, Singapore, pp 293–302
Manasrah AM, Gupta BB (2019) An optimized service broker routing policy based on differential evolution algorithm in fog/cloud environment. Clust Comput 22(1):1639–1653
Mebrek A, Merghem-Boulahia L, Esseghir M (2017) Efficient green solution for a balanced energy consumption and delay in the IoT-Fog-Cloud computing. In: 2017 IEEE 16th international symposium on network computing and applications (NCA). IEEE, pp 1–4. https://doi.org/10.1109/NCA.2017.8171359
Millham R, Agbehadji IE, Frimpong SO (2021) The paradigm of fog computing with bio-inspired search methods and the "5Vs" of big data. In: Fong S, Millham R (eds) Bio-inspired algorithms for data streaming and visualization, big data management, and fog computing. Springer tracts in nature-inspired computing. Springer, Singapore, pp 145–167
Mohammadi A, Rezvani MH (2019) A novel optimized approach for resource reservation in cloud computing using producer–consumer theory of microeconomics. J Supercomput 75(11):7391–7425
Ning Z, Dong P, Wang X, Hu X, Liu J, Guo L, Hu B, Kwok R, Leung VC (2020) Partial computation offloading and adaptive task scheduling for 5G-enabled vehicular networks. IEEE Trans Mob Comput. https://doi.org/10.1109/TMC.2020.3025116
Parvizi E, Rezvani MH (2020) Utilization-aware energy-efficient virtual machine placement in cloud networks using NSGA-III metaheuristic approach. Cluster Comput 23:2945–2967
Qiu X, Xu JX, Xu Y, Tan KC (2017) A new differential evolution algorithm for minimax optimization in robust design. IEEE Trans Cybern 48(5):1355–1368
Salaht FA, Desprez F, Lebre A (2020) An overview of service placement problem in fog and edge computing. ACM Comput Surv (CSUR) 53(3):1–35
Seada H, Deb K (2015) U-NSGA-III: a unified evolutionary algorithm for single, multiple, and many-objective optimization. In: Gaspar-Cunha A, Henggeler Antunes C, Coello C (eds) Evolutionary multi-criterion optimization. EMO 2015. Lecture notes in computer science, vol 9019. Springer, Cham. https://doi.org/10.1007/978-3-319-15892-1_3
Shahidinejad A, Ghobaei-Arani M (2020) Joint computation offloading and resource provisioning for edge-cloud computing environment: a machine learning-based approach. Softw Pract Exp 50:2212–2230
Shah-Mansouri H, Wong VW (2018) Hierarchical fog-cloud computing for IoT systems: a computation offloading game. IEEE Internet Things J 5(4):3246–3257
Shakarami A, Ghobaei-Arani M, Masdari M, Hosseinzadeh M (2020) A survey on the computation offloading approaches in mobile edge/cloud computing environment: a stochastic-based perspective. J Grid Comput 18:639–671
Subramaniam EVD, Krishnasamy V (2021) Energy aware smartphone tasks offloading to the cloud using gray wolf optimization. J Ambient Intell Human Comput 12:3979–3987
Sun M, Xu X, Tao X et al (2020) Large-scale user-assisted multi-task online offloading for latency reduction in D2D-enabled heterogeneous networks. IEEE Trans Netw Sci Eng 7(4):2456–2467
Tavakoli-Someh S, Rezvani MH (2019) Multi-objective virtual network function placement using NSGA-II meta-heuristic approach. J Supercomput 75(10):6451–6487
Tušar T, Filipič B (2007) Differential evolution versus genetic algorithms in multiobjective optimization. In: Obayashi S, Deb K, Poloni C, Hiroyasu T, Murata T (eds) Evolutionary multi-criterion optimization. EMO 2007. Lecture notes in computer science, vol 4403. Springer, Berlin, Heidelberg, pp 257–271
Wang J, Lv T, Huang P, Mathiopoulos PT (2020) Mobility-aware partial computation offloading in vehicular networks: a deep reinforcement learning based scheme. China Commun 17(10):31–49
Yang T, Feng H, Yang C et al (2018) Multivessel computation offloading in maritime mobile edge computing network. IEEE Internet Things J 6(3):4063–4073
Yassine A, Singh S, Hossain MS et al (2019) IoT big data analytics for smart homes with fog and cloud computing. Future Gener Comput Syst 91:563–573
Yu Y, Bu X, Yang K, Wu Z, Han Z (2018) Green large-scale fog computing resource allocation using joint benders decomposition, Dinkelbach algorithm, ADMM, and branch-and-bound. IEEE Internet Things J 6(3):4106–4117
Zhang G, Shen F, Liu Z et al (2019) FEMTO: fair and energy-minimized task offloading for fog-enabled IoT networks. IEEE Internet Things J 6(3):4388–4400. https://doi.org/10.1109/JIOT.2018.2887229

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations'' (in PDF at the end of the article below the references.